Дипломная работа

Разработка программного средства анализа движения транспорта по видеозаписям

Сверточная нейронная сеть и анализ транспортного потока

Тема задания

CNN- Տեսաշարերի միջոցով տրանսպորտային միջոցների շարժման և երթևկության հոսքի վերլուծության ծրագրային միջոցի մշակում

Реализация на Python

Автор _____

Руководитель _____

Учебное заведение _____

2026

Содержание

Аннотация

Введение

1 Анализ задачи и методов

2 Подготовка корпуса

3 Проектирование и реализация

4 Протокол экспериментальной проверки

5 Экспериментальные результаты

6 Руководство пользователя

Заключение

Список использованных источников

Приложение А Состав экспортируемых данных

Приложение Б Воспроизведение экспериментов

Аннотация

Работа посвящена разработке программного средства на Python для обнаружения транспортных средств, восстановления их траекторий и подсчета пересечений контрольной линии. Детектор построен на сверточной архитектуре YOLOv8n. Для межкадрового сопоставления применяется ByteTrack; дополнительный программный слой поддерживает непрерывность коротких фрагментов траектории и стабилизирует класс объекта по истории наблюдений. Пользовательский интерфейс позволяет загрузить видеозапись, выбрать обученную модель и положение линии, посмотреть размеченное видео и выгрузить статистику.

Подготовлен корпус из пяти классов транспорта на основе VisDrone2019-DET. Исходные архивы проверены, разметка преобразована в формат YOLO, точные дубликаты исключены. Проведено сравнение случайной и предобученной инициализации на общей пилотной выборке. Основной эксперимент использует полный подготовленный train. Результаты обнаружения и проверки подсчета рассматриваются отдельно: правильное общее число пересечений может скрывать одновременные пропуски и повторный учет.

Ключевые слова: сверточная нейронная сеть, обнаружение объектов, транспортный поток, YOLOv8, ByteTrack, Python, видеозапись.

Введение

Видеозапись дорожной сцены содержит информацию о положении и движении транспортных средств. Для превращения этой информации в количественную статистику необходимо решить несколько связанных задач. Сначала на каждом кадре требуется обнаружить машины и определить их классы. Затем наблюдения одной физической машины необходимо связать во времени. Наконец, по полученной траектории нужно определить событие, которое будет учитываться в статистике. В настоящей работе таким событием является пересечение заданной контрольной линии.

Простое суммирование числа обнаруженных объектов по кадрам не решает задачу подсчета транспортного потока. Один автомобиль может находиться в поле зрения камеры много секунд и быть обнаружен сотни раз. Перекрытие соседними машинами, изменение масштаба и нестабильность рамки создают разрывы наблюдения. Поэтому программное средство должно работать с последовательностью кадров и хранить состояние траекторий.

Цель работы — разработать на Python средство анализа движения транспортных средств по видеозаписям, обучить CNN-детектор на размеченном корпусе и проверить качество обнаружения и учета пересечений. Объект исследования — видеозаписи дорожных сцен и изображения с разметкой транспорта. Предмет исследования — сочетание сверточного обнаружения объектов, межкадрового сопоставления и геометрического правила подсчета.

Задачи работы включают подготовку данных, построение воспроизводимого эксперимента обучения, реализацию обработки видео, разработку интерфейса и экспорт результатов. Эксперимент должен сохранять не только итоговые значения метрик, но и параметры запуска, версии библиотек, веса, графики и сведения об источниках данных. Это позволяет проверить связь между выводами и фактически выполненными вычислениями.

Практический результат — локальное приложение для обработки готовых видеозаписей. Оно ориентировано на исследовательское использование и демонстрацию методов компьютерного зрения. Полученные показатели не рассматриваются как подтверждение пригодности для регулирования реального движения: для такой оценки необходимы дополнительные записи целевых камер, независимая разметка и проверка устойчивости в разных условиях.

1 Анализ задачи и методов

1.1 Требования к анализу транспортного потока

Входом системы является видеофайл с постоянной частотой кадров. Пользователь задает модель, порог уверенности и контрольную линию. Выход включает размеченное видео, таблицу пересечений, покадровые наблюдения, характеристики траекторий и агрегированную статистику. Для каждой записи в таблице пересечений должны быть доступны номер кадра, время, идентификатор, класс транспорта и направление перехода.

Программа должна учитывать конечную длину контрольной линии. Пересечение ее бесконечного продолжения за пределами выбранного участка не является событием. Небольшие колебания опорной точки около линии не должны создавать серию пересечений. Если наблюдения разделены большим промежутком времени, соединять их прямым отрезком для учета события нельзя: фактическое движение в пропущенном интервале неизвестно.

Дополнительные требования относятся к проверяемости результата. Исходное видео должно сохраняться, повторный запуск не должен перезаписывать готовые результаты, а параметры обработки необходимо записывать вместе со сводкой. Ошибки открытия файла, некорректная частота кадров и недопустимые координаты линии должны приводить к понятному сообщению.

1.2 Сверточные признаки и обнаружение объектов

Сверточный слой применяет обучаемые фильтры к локальным участкам изображения. Общие веса фильтра используются в разных пространственных позициях, поэтому сеть может выделять повторяющиеся признаки независимо от конкретного положения объекта. Последовательность таких преобразований строит представления от локальных границ и текстур до более сложных сочетаний признаков. Обучение корректирует параметры по ошибке предсказания на размеченных примерах.

Классификация изображения и обнаружение объектов различаются по выходу модели. Классификатор присваивает метку всему изображению, тогда как детектор возвращает несколько объектов, их рамки, классы и уверенность. Для дорожной сцены требуется именно обнаружение: в одном кадре могут одновременно находиться автомобили, автобусы и мотоциклы разного размера.

Малые объекты представляют отдельную сложность. Если дальняя машина занимает лишь несколько пикселей, уменьшение изображения до входного размера сети может удалить различимые детали. Повышение размера входа сохраняет больше пространственной информации, но увеличивает вычислительную нагрузку. Поэтому размер входа является параметром эксперимента, а не только техническим свойством интерфейса.

1.3 Выбор YOLOv8n

В проекте использована компактная модель YOLOv8n из библиотеки Ultralytics [3]. Ее применение позволяет обучать и выполнять детекцию на доступном локальном компьютере. Выбор компактной модели связан с ограничением памяти и времени вычислений. Он не доказывает, что эта архитектура оптимальна для всех дорожных сцен.

В модели можно выделить извлечение признаков, объединение признаков разных масштабов и выходные компоненты обнаружения. Многомасштабное представление необходимо потому, что машины имеют разные размеры в зависимости от расстояния до камеры.

Предсказанные рамки проходят подавление перекрывающихся обнаружений, после чего используются для отслеживания.

CNN-архитектура и библиотечные функции обучения являются готовыми компонентами. Собственный вклад проекта состоит в подготовке транспортной задачи, организации экспериментов, обработке видеопоследовательности, правилах учета, дополнительных проверках непрерывности, экспорте и пользовательском интерфейсе. Эти части не следует смешивать с заявлением о разработке новой архитектуры нейронной сети.

1.4 Обучение с нуля и перенос обучения

При случайной инициализации параметры сети обучаются без предварительно загруженных весов. Модель должна освоить общие визуальные признаки и конкретные классы целевого корпуса в пределах заданного бюджета. При дообучении стартовые веса получены на COCO, после чего параметры изменяются на транспортных данных. Дообучение также является обучением, поскольку целевая функция и градиентные обновления применяются к новому размеченному корпусу.

Для сопоставления этих режимов необходимо фиксировать архитектуру, данные, размер входа, число эпох и прочие параметры. Сравнение модели на маленькой выборке с моделью на полном корпусе не позволяет объяснить изменение качества только начальной инициализацией. Поэтому пилотное сравнение выделено в отдельный эксперимент, а обучение на полном train рассматривается как самостоятельный этап.

Результат зависит от случайности и доступного бюджета. Единственный запуск с seed 42 позволяет воспроизвести основные настройки, но не дает доверительный интервал качества. Некоторые операции на MPS не имеют полностью детерминированной реализации; библиотека выводит соответствующие предупреждения. Эти ограничения фиксируются в материалах эксперимента.

1.5 Отслеживание и ByteTrack

Отслеживание связывает рамки одного объекта между кадрами и назначает идентификатор. ByteTrack использует высокоуверенные обнаружения для основного сопоставления и может восстанавливать существующие траектории с помощью обнаружений меньшей уверенности [4]. Такой подход полезен при временном перекрытии, когда детектор продолжает видеть объект, но его оценка уверенности падает.

Из этого следует требование к программному конвейеру: нельзя заранее отбрасывать все обнаружения ниже пользовательского порога создания трека. В реализации нижний порог детектора отделен от порога инициализации. Низкая уверенность допускается для восстановления существующей траектории, а новая траектория создается по более строгому условию. Конкретные пороги сохраняются в tracker.yaml и summary.json.

Идентификатор алгоритма отслеживания не равен гарантированно идентичности физической машины. Новый ID может появиться после перекрытия, а неверное сопоставление может связать разные машины. Поэтому качество обнаружения, устойчивость идентификаторов и точность пересечений требуют раздельной проверки.

2 Подготовка корпуса

2.1 Источник и целевые классы

В качестве открытого размеченного корпуса выбран VisDrone2019-DET [1, 2]. Используются официальные train, val и test-dev с доступными аннотациями. Корпус содержит съемку с беспилотных аппаратов. Это дает разнообразие масштаба и плотности объектов, однако создает различие с ракурсом стационарной дорожной камеры.

Целевая задача содержит пять классов: легковой автомобиль car, фургон van, грузовик truck, автобус bus и мотоцикл motorcycle. Последний соответствует исходному классу motor. Остальные категории не включены. Такое преобразование уменьшает множество классов до транспорта, необходимого для выбранной демонстрации, но меняет исходный протокол оценки.

Игнорируемые области исходного корпуса не превращаются в целевые рамки и не маскируются на изображениях. Следовательно, модель и библиотечная оценка могут реагировать на объекты в этих областях иначе, чем официальный оценщик VisDrone. Значения mAP в работе относятся к адаптированной задаче пяти классов и не должны сравниваться напрямую с официальными результатами соревнования.

2.2 Преобразование разметки

Подготовка реализована в scripts/prepare_visdrone.py. Архивы проходят проверку целостности, изображения проверяются на возможность чтения. Для каждой аннотации исходные координаты прямоугольника переводятся в границы рамки и ограничиваются

размерами изображения. Рамки с нулевой площадью исключаются. После сопоставления классов координаты нормируются.

Формат YOLO содержит строку для каждого объекта: номер класса, координаты центра, ширину и высоту рамки. Все четыре геометрические величины выражены относительно ширины и высоты изображения. Такая запись не зависит от исходного разрешения. Пустой файл меток означает отсутствие целевых рамок после преобразования, а не ошибку чтения изображения.

Обрезка рамки необходима, если исходный объект частично выходит за кадр. Без нее нормированные координаты могут выйти за допустимый диапазон. Проверка на синтетическом архиве охватывает эту ситуацию, сохранение нужных классов, исключение игнорируемых аннотаций и обработку дубликатов.

2.3 Разбиение и контроль дубликатов

Официальное разбиение сохранено. Проверочная выборка используется для выбора лучшей эпохи и порядка рекомендуемых моделей. Тестовая выборка используется для итоговой оценки завершенного запуска. Тестовые изображения не участвуют в обновлении весов и не определяют порог интерфейса.

Для поиска точных дубликатов вычисляется SHA-256 содержимого изображений. При совпадении сохраняется экземпляр с приоритетом test, затем val, затем train. Исключены три точных повтора внутри исходных выборок. После этой операции в подготовленных данных нет повторов с одинаковыми байтами между выборками. Это не является доказательством независимости всех сцен: похожие кадры и общие камеры требуют отдельной проверки.

Пилотная выборка содержит 1024 изображения train. Отбор с фиксированным seed 42 организован с чередованием префиксов сцен, чтобы не брать только начало списка. Полная выборка содержит 6469 изображений train. Во всех экспериментах val содержит 547 изображений, test — 1610 после удаления точных повторов.

2.4 Дисбаланс классов

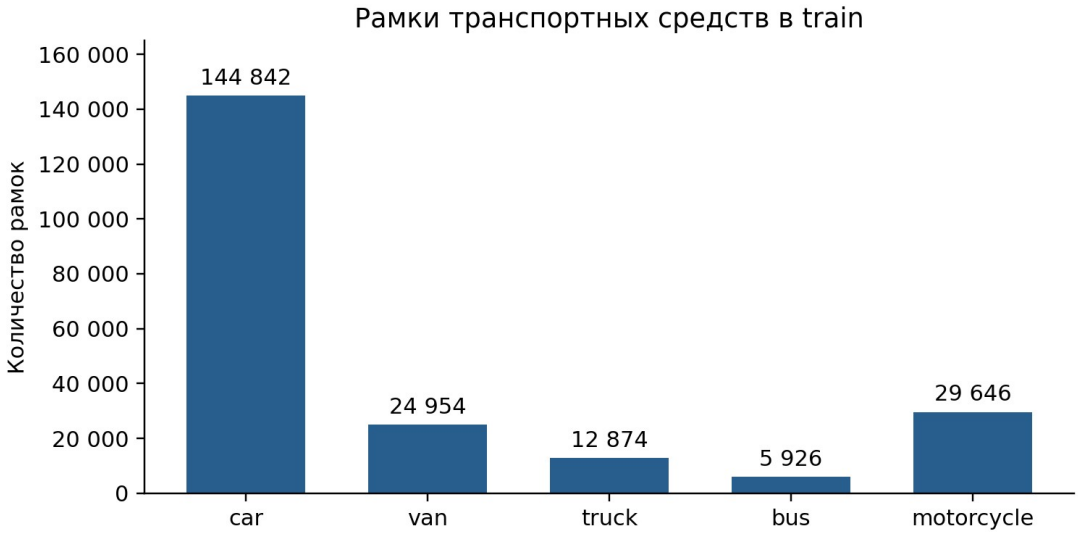


Рисунок 1 Распределение рамок по классам полного train

В train размечено 218242 рамки транспорта. Из них 144842 относятся к car, 24954 к van, 12874 к truck, 5926 к bus и 29646 к motorcycle. Число примеров автобусов значительно меньше числа легковых машин. Общая метрика должна поэтому дополняться оценками по классам: высокая точность частого класса не означает, что редкие категории распознаются достаточно хорошо.

Помимо числа объектов, на качество влияет их размер, перекрытие и условия съемки. В настоящем эксперименте не проведено отдельное ранжирование по размеру и погоде. Нельзя выводить устойчивость к ночной съемке или дождю только из общей метрики на корпусе. Такие условия следует проверять на отдельной группе видеозаписей.

3 Проектирование и реализация

3.1 Архитектура программы

Программное средство разделено на подготовку данных, эксперимент обучения, анализ видео и интерфейс. Скрипт подготовки формирует изображения, метки, конфигурации и отчет о корпусе. Скрипт эксперимента обучает модель, выбирает сохраненные лучшие веса по validation и выполняет оценку на val и test. Анализатор использует локальный файл весов, а интерфейс читает результаты завершенных экспериментов.

Такое разделение позволяет запускать обучение без открытия интерфейса и повторно обрабатывать видео без повторного обучения. Наличие отдельного отчета для каждого запуска сохраняет происхождение модели. При выборе в интерфейсе показывается название эксперимента, а не только общий тип инициализации.

Основные модули — traffic_cnn/cli.py для команд, counting.py для пересечений и tracklets.py для непрерывности коротких фрагментов. app.py содержит интерфейс Streamlit. Конфигурации данных хранятся в configs, вычислительные артефакты — в runs и outputs, отчеты — в reports. Рабочие материалы диплома отделены в thesis.

3.2 Обработка видеозаписи

Анализатор проверяет наличие видео и локальных весов, открывает видео через OpenCV и считывает частоту кадров и размеры. Значения должны быть конечными и положительными. Координаты контрольной линии должны находиться внутри кадра. Если каталог содержит готовые результаты, запуск прекращается, чтобы не уничтожить предыдущий эксперимент.

Кадры обрабатываются последовательно. YOLOv8n возвращает рамки, после чего ByteTrack связывает их с идентификаторами. Для каждого наблюдения вычисляется середина нижней стороны рамки. Эта точка выбрана как приближение положения транспортного средства относительно дорожной поверхности, но она не является точным физическим контактом колес с дорогой.

По наблюдениям строятся короткие отображаемые траектории. События записываются в таблицу, а накопленный счет выводится на кадр. Размеченное видео сохраняется с исходной частотой кадров. При доступности кодека H.264 используется avc1 для воспроизведения в браузере; иначе применяется mp4v с возможностью скачивания результата.

3.3 Непрерывность и стабилизация класса

Экспериментальный режим доступен отдельно от стандартного ByteTrack, используемого по умолчанию. Дополнительный слой сохраняет соответствие исходного ID и канонического ID. Если появился новый ID, он может быть связан только с недавно потерянным фрагментом, отсутствующим среди активных объектов текущего кадра. Предыдущее положение рамки прогнозируется линейно по последнему изменению координат. Сопоставление допускается при IoU не меньше 0.5 и промежутке не больше десяти кадров.

При близких конкурирующих кандидатах сопоставление отвергается. Если два новых обнаружения претендуют на один потерянный фрагмент, оба сохраняют самостоятельные идентификаторы. Эти условия снижают риск объединения соседних машин, но не устраняют его полностью. Простое пространственное совпадение не заменяет полноценное распознавание внешнего вида.

Класс машины определяется суммой уверенностей наблюдений по категориям. Разовое переключение car на van или truck на bus поэтому не обязательно меняет итоговую категорию траектории. Длительная ошибочная история может, наоборот, удерживать неверный класс. Выбранное правило является инженерной эвристикой и проверяется как часть конвейера, а не заявляется новым алгоритмом распознавания.

3.4 Геометрия пересечения

Пусть A и B задают ориентированный отрезок, P — опорная точка. Знак векторного произведения направления AB и вектора AP определяет сторону линии. После деления на длину AB получается знаковое расстояние до прямой. Зона шириной четыре пикселя около линии игнорируется, чтобы небольшие колебания точки не создавали переходы.

Для каждого ID сохраняется последнее положение вне зоны нечувствительности. При смене стороны определяется точка пересечения отрезка между старым и новым положением с прямой AB. Событие создается только если эта точка лежит между A и B и временной разрыв не превышает тридцать кадров. Один канонический ID учитывается не больше одного раза за весь ролик.

Для горизонтальной линии слева направо отрицательно-положительный переход означает движение к нижнему краю кадра. Противоположный переход соответствует движению к верхнему краю. Эти названия относятся к изображению; они не являются географическими направлениями. В консольной версии может быть задан произвольный ориентированный отрезок.

Машина, впервые обнаруженная уже за линией, не имеет наблюдаемого перехода и не учитывается. Программа не восстанавливает такое событие предположительно. Если автомобиль разворачивается и пересекает линию повторно с тем же ID, второй переход не добавляется. Это явно заданное правило уникального учета в пределах записи; для иной прикладной задачи потребуется другой режим.

3.5 Характеристики движения и потока

Интенсивность выражается формулой q = 3600 × N / T, где N — число учтенных переходов, T — длительность видео в секундах. Полученное значение имеет единицы машин в час. Для короткой записи оно является пересчетом наблюдаемого интервала, а не фактически измеренным числом машин за целый час.

В таблице траекторий сохраняются смещение по x и y, длительность наблюдения и сумма расстояний между соседними точками. Отношение этой суммы к длительности выражается в пикселях в секунду. Оно зависит от перспективы, масштаба и колебаний рамки, поэтому не интерпретируется как дорожная скорость. Для измерения километров в час требуется калибровка положения камеры и реальных расстояний.

Покадровая статистика содержит число видимых отслеживаемых машин и накопленный счет. По ней можно увидеть изменение наблюдаемой загрузки кадра и моменты пересечений. Среднее число видимых машин не преобразуется в плотность машин на километр: программа не знает физическую длину видимого дорожного участка.

3.6 Интерфейс и экспорт

Интерфейс содержит вкладки анализа видео, результатов обучения и данных. Пользователь может выбрать демонстрационный ролик либо загрузить свой файл, задать положение горизонтальной линии, порог и вычислительное устройство. Во время обработки показываются число прочитанных кадров и текущий счет. Журнал сохраняется в каталоге запуска для диагностики.

Завершенный результат содержит видео, crossings.csv, tracks.csv, tracks_summary.csv, frame_stats.csv и summary.json. Таблица crossings.csv предназначена для проверки отдельных событий. tracks.csv хранит исходный и канонический ID вместе с координатами и классом. tracks_summary.csv содержит характеристики движения, а frame_stats.csv позволяет построить временные графики.

В интерфейсе доступны скачивание этих файлов и просмотр сводки. Модели упорядочены по validation mAP50–95. Данные test показываются для описания эксперимента и не используются при выборе порядка. Сервер запускается локально на 127.0.0.1; загрузка видео означает передачу файла локальному процессу приложения.

4 Протокол экспериментальной проверки

4.1 Метрики обнаружения

Precision характеризует долю правильных обнаружений среди предсказанных, recall — долю найденных объектов среди размеченных. Сопоставление рамок зависит от пересечения относительно объединения IoU. Ошибка локализации может привести к отрицательному результату, даже если класс объекта назван верно.

AP определяется по зависимости precision от recall при изменении порога уверенности. mAP50 усредняет AP по классам при IoU 0.50. mAP50–95 дополнительно усредняет значения при порогах IoU от 0.50 до 0.95 с шагом 0.05. Более строгая метрика сильнее реагирует на неточность рамок. Библиотечные precision и recall в отчете не следует считать значениями при фиксированном пользовательском пороге 0.3.

Все значения сохраняются с параметрами оценки и детализацией по категориям. Общий mAP дополняется графиками обучения и примерами предсказаний. Метрики относятся к подготовленному набору изображений. Они не измеряют устойчивость идентификаторов и не определяют точность подсчета по видео.

4.2 Проверка событий

Для видео недостаточно сравнить итоговые суммы. Если одна машина пропущена, а другая учтена дважды, абсолютная ошибка общего счета может быть нулевой. Поэтому проводится взаимно однозначное сопоставление событий по направлению и времени. Каждому опорному переходу соответствует не более одного предсказанного события.

Первый визуальный ориентир содержит временные интервалы пересечений, поскольку контактные листы не дают точного номера каждого события. Сопоставление использует эти интервалы с допуском шесть кадров. Сначала максимизируется число допустимых соответствий, затем учитывается согласие класса. Непокрытые интервалы, лишние события и ошибки категории записываются отдельно.

Ориентир подготовлен ассистентом по кадрам до просмотра предсказаний и требует независимого подтверждения человеком. Он служит для диагностики конкретного примера. Приведенные совпадения не являются валидированным показателем общей точности.

Первый ролик также использовался при анализе ошибок конвейера, поэтому проверку его доработки следует считать проверкой на разработочном примере.

4.3 Программные проверки

Автоматические тесты проверяют конечный отрезок, направление, нечувствительность около линии, однократный учет, пропущенные интервалы и недопустимые параметры. Для непрерывности дополнительно проверяются замена ID после короткой потери, одновременные соседние объекты, неоднозначные кандидаты и конкуренция за потерянный фрагмент.

Тест преобразования данных использует небольшой специально созданный архив с известными аннотациями. Тест видео проверяет возможность чтения и записи, число обработанных кадров, длительность и наличие таблиц. Проверка интерфейса подтверждает открытие страницы без исключений. Эти тесты не подменяют эксперимент на реальных изображениях и не доказывают точность обнаружения транспорта.

4.4 Быстродействие и воспроизводимость

Рабочая машина — Apple M4 с 16 ГБ памяти. Обучение использует MPS. Время запуска измеряется отдельно от оценки на test. Анализ видео может выполняться на CPU или MPS, причем результаты замера зависят от размера входа, разрешения вывода, кодирования и параллельной нагрузки.

Для сравнения скорости необходимо одинаковое видео, одинаковое устройство и отсутствие конкурирующего обучения. Замер во время работы GPU не сопоставим напрямую с замером на свободном компьютере. Число кадров в секунду описывает полный текущий конвейер, а не только вычисление CNN. Не заявляется гарантированная работа в реальном времени для любого разрешения.

Каждый эксперимент сохраняет конфигурацию, число фактически завершенных эпох, seed, версии Python, PyTorch и Ultralytics, путь к лучшим весам и результаты по классам. SHA-256 архивов и исходных файлов обеспечивает контроль целостности. Сохранение хешей не гарантирует побитовую идентичность обучения на другой платформе.

5 Экспериментальные результаты

5.1 Обучение детектора

Пилотное сравнение выполнено на 1024 изображениях train при размере входа 416, batch 8 и 15 эпохах. Для обеих инициализаций использованы одинаковые выборки и seed 42. Это сопоставление относится к ограниченному бюджету, а не к полностью сошедшимся моделям.

Основной запуск vehicle-finetune-full использует полный подготовленный train из 6469 изображений. Инициализация: COCO-pretrained. Завершено 10 эпох из 10; размер входа 640, batch 8, устройство mps. Время обучения 69.1 минуты. Проверочная и тестовая выборки совпадают с пилотным экспериментом.

На test получены mAP50 0.3441 и mAP50–95 0.2029. Precision составляет 0.4607, recall — 0.3583. Эти значения характеризуют библиотечную оценку подготовленной задачи и не являются точностью учета физических машин.

Зафиксированная длительность основного запуска включает длительные перерывы. Она не характеризует непрерывную вычислительную производительность и не сопоставима напрямую с временем пилотных запусков.

Запуск	Precision	Recall	mAP50	mAP50–95
COCO full	0.4607	0.3583	0.3441	0.2029
COCO pilot	0.2559	0.1942	0.1384	0.0686
С нуля pilot	0.4878	0.0378	0.0262	0.0087

В пилотном эксперименте дообучение дает более высокие mAP50 и mAP50–95, однако абсолютные значения остаются низкими. Один seed и ограниченное число эпох не определяют максимальные возможности архитектуры. Различия основного и пилотного запусков включают объем train и размер входа; изменение качества нельзя приписывать только одному из этих факторов.

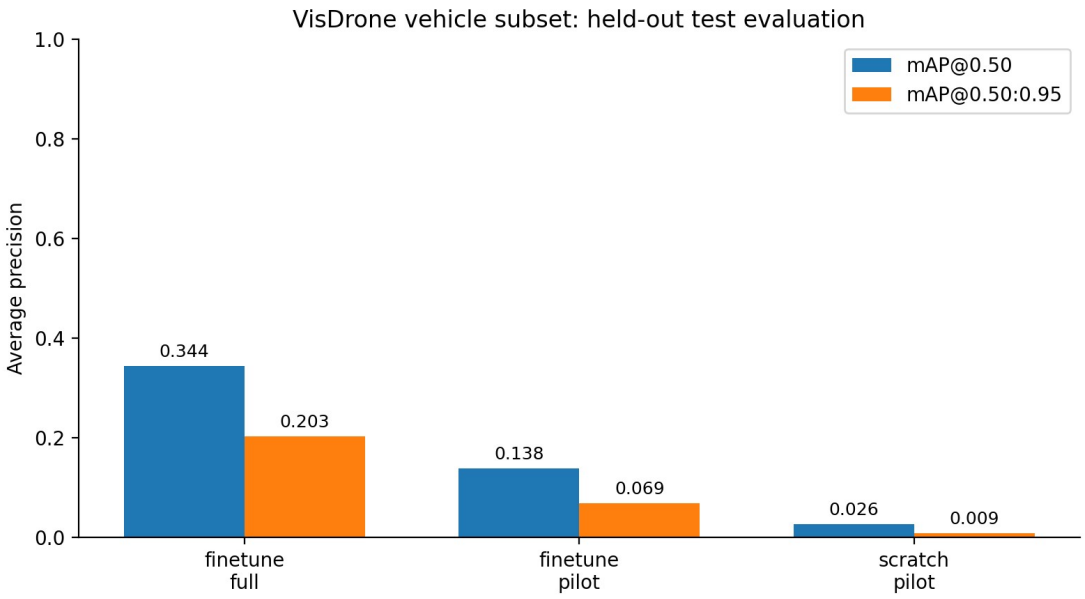


Рисунок 2 Тестовые метрики завершенных экспериментов

Рекомендуемая модель выбрана по validation mAP50–95: vehicle-finetune-full, значение 0.2205. Тестовые значения не используются для выбора. Метрики категорий этой модели приведены ниже.

Класс	Precision	Recall	mAP50	mAP50–95
car	0.5927	0.6371	0.6137	0.3598
van	0.3556	0.2679	0.2537	0.1543
truck	0.4611	0.2561	0.2623	0.1568
bus	0.5128	0.4663	0.4456	0.2915
motorcycle	0.3816	0.1639	0.1453	0.0520

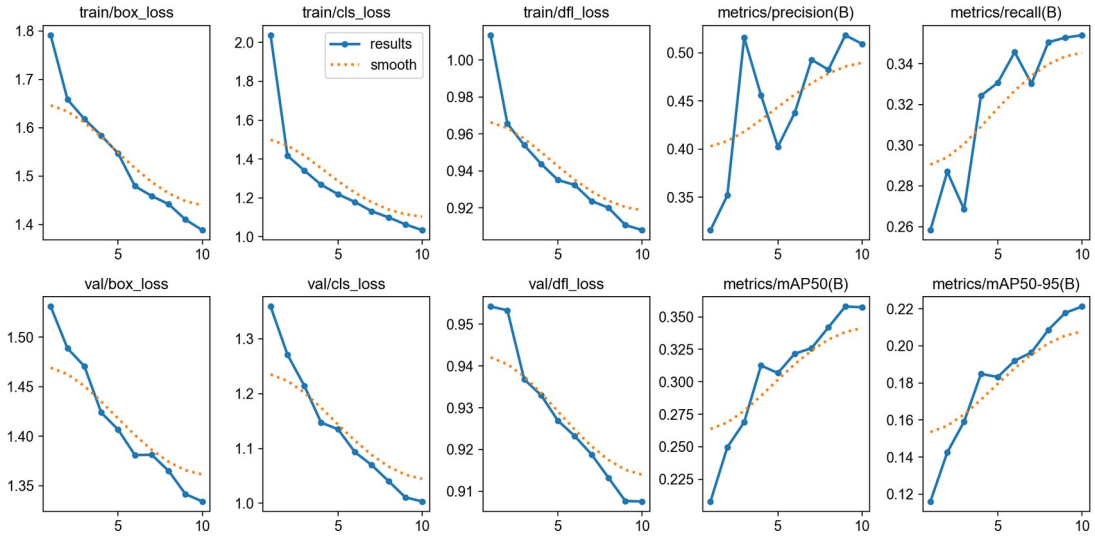


Рисунок 3 Изменение потерь и проверочных метрик рекомендуемой модели

5.2 Проверка транспортных событий

Запуск	Событий	Совпало	Пропуск	Лишние
finetune-tracking-v2	10	9	1	1
finetune	10	9	1	1
full-baseline	10	9	1	1
full-continuity	9	8	2	1
scratch	11	10	0	1

Первый дорожный пример длится 21.52 секунды и содержит предварительный ориентир из десяти пересечений. В исходном пилотном дообучении общий счет десять сопровождался девятью совпадениями, одним пропуском и одним лишним событием. Пропуск первого грузовика и позднее повторное обнаружение другого грузовика компенсировались в сумме. Это подтверждает необходимость проверки отдельных событий.

Модификация отслеживания оценивается на том же разработочном примере, поэтому ее результат не является независимой тестовой оценкой. Сравнение полного детектора с исходным и доработанным отслеживанием использует одинаковое видео, линию, порог и

веса. Значения берутся из сохраненных таблиц событий. Визуальный ориентир подготовлен ассистентом и требует подтверждения человеком.

Стандартный вариант дал 9 совпадений, экспериментальный — 8. Дополнительные правила не обеспечили улучшения на этом примере. В пользовательском интерфейсе и CLI стандартное отслеживание выбрано по умолчанию; модификация оставлена отдельным исследовательским режимом. Вывод ограничен предварительной разметкой разработочного ролика.

Запуск demo-full-baseline: 10 пересечений, направление к нижнему краю 6, к верхнему краю 4. Скорость текущего запуска 11.89 кадра/с; это измерение полного конвейера с записью видео.

Запуск demo-full-continuity: 9 пересечений, направление к нижнему краю 5, к верхнему краю 4. Скорость текущего запуска 11.92 кадра/с; это измерение полного конвейера с записью видео.

Запуск demo-full-second: 634 пересечений, направление к нижнему краю 361, к верхнему краю 273. Скорость текущего запуска 16.34 кадра/с; это измерение полного конвейера с записью видео.

Запуск demo-full-second-baseline: 659 пересечений, направление к нижнему краю 369, к верхнему краю 290. Скорость текущего запуска 19.01 кадра/с; это измерение полного конвейера с записью видео.

5.3 Дополнительный видеопример и ограничения

Дополнительно обработан второй пример Roboflow Supervision vehicles-2.mp4, ранее не использовавшийся для доработки правил. Его длительность 42.54 секунды, число кадров 1275, разрешение 1920 на 1080. Получено 634 событий. Этот запуск проверяет выполнение конвейера на другой записи; без независимой разметки число событий не является оценкой точности.

Экспериментальный режим второго примера создал 2390 ID и 634 событий; стандартный — 2515 ID и 659 событий. Эти величины нельзя отождествлять с числом физических автомобилей. Без разметки сравнение сумм не определяет лучший режим. Реальный временной масштаб исходной съемки, включая возможное ускорение или монтаж, не

подтвержден. Пересчет по длительности файла не является подтвержденной интенсивностью реального дорожного потока.

Нельзя делать вывод о надежности в реальном городе по одной сцене и адаптированному VisDrone. Выборки не дают независимого контроля погодных условий и камер, а точная дедупликация не удаляет все похожие сцены. Разметка подсчета пока предварительная. Физическая скорость и дорожная плотность не измерены. Программная работоспособность и статистическая обоснованность точности имеют разные границы.

6 Руководство пользователя

6.1 Установка и запуск

Для проверенного окружения используется Python 3.10. Сначала создается виртуальное окружение, затем устанавливается проект и зависимости. Файл requirements-lock.txt фиксирует версии проверенного окружения. Установка этих версий на другую аппаратную платформу может зависеть от наличия совместимых бинарных пакетов.

Локальное приложение запускается файлом run_app.command на macOS либо командой python -m streamlit run app.py с адресом сервера 127.0.0.1. В браузере открывается http://127.0.0.1:8501. Чтобы получить новые результаты обучения в открытом интерфейсе, используется кнопка обновления.

Для обработки своего видео нужно выбрать источник, загрузить файл, выбрать модель и положение линии. Линию следует разместить в области, где машины видны до и после пересечения. Слишком близкая к границе кадра линия повышает риск пропусков, поскольку объект может быть обнаружен только с одной стороны. Перед основным запуском полезно проверить положение на коротком фрагменте.

6.2 Интерпретация результата

После обработки нужно просмотреть видео и таблицу пересечений. Итоговый счет, число треков и число физических машин — разные величины. Один ID может соответствовать неверно объединенной траектории, а одна машина может получить несколько ID. При сомнении следует найти кадр события в crossings.csv и сопоставить его с исходной записью.

Порог влияет на начало новых треков. Его уменьшение может находить слабые обнаружения, но также увеличивать ложные объекты. Низкоуверенные наблюдения

существующего трека обрабатываются отдельным нижним порогом. Выбирать пользовательский порог нужно по проверочному видео, а не по желаемому итоговому счету.

Экспортированные данные позволяют повторно построить временные графики и проверить направления. Пиксельное перемещение используется для анализа видимой траектории. Числовое значение в пикселях в секунду нельзя переносить между машинами на разных расстояниях от камеры как физическую скорость.

6.3 Обучение на собственных данных

Для новой камеры желательно собрать отдельные изображения и разметить целевые рамки. Train, val и test нужно разделять по исходным записям и камерам, чтобы соседние кадры не попадали в разные выборки. В конфигурации YOLO задаются пути и порядок категорий. Изменение порядка классов требует согласования с метками и не может быть выполнено только переименованием файла весов.

После обучения модель оценивается на независимом test, затем отдельно проверяется подсчет по видео. Файл best.pt выбирается по validation. Результаты текущего исследования не задают универсальное число эпох: графики могут показать, что обучение следует продолжить, изменить данные или выбрать другой размер входа.

Заключение

Основной опыт завершен на полном train из 6469 изображений. Тестовые mAP50 и mAP50–95 составили 0.3441 и 0.2029. Результат измерен после десяти эпох дообучения; он не устанавливает предельное качество модели или надежность на другой камере.

Разработано программное средство, объединяющее CNN-детекцию, межкадровое отслеживание, геометрический учет пересечений и локальный интерфейс. Подготовлен транспортный корпус с проверкой целостности и сохраненными источниками. Реализованы обучение и оценка моделей, экспорт событий, траекторий и покадровой статистики. Дополнительные правила поддерживают короткие разрывы идентификаторов и стабилизируют категории по истории.

Эксперименты показывают необходимость разделять качество детектора и качество подсчета. Совпадение общей суммы с визуальным ориентиром не исключает ошибок отдельных событий. Оценки на съемке с дрона не подтверждают перенос на любую

дорожную камеру. Одного seed и короткой демонстрации недостаточно для установления общей надежности системы.

Дальнейшая проверка должна включать независимую человеческую разметку видеозаписей целевой камеры, разные условия освещения и интенсивности движения, анализ малых и перекрытых объектов. При необходимости измерения физической скорости требуется калибровка. Эти направления относятся к расширению экспериментальной проверки и не заменяются корректным выполнением программных тестов.

Список использованных источников

[1] VisDrone Dataset. Официальный репозиторий. https://github.com/VisDrone/VisDrone-Dataset

[2] Zhu P. и др. Detection and Tracking Meet Drones Challenge. https://arxiv.org/abs/2001.06303

[3] Ultralytics. YOLOv8. Документация модели. https://docs.ultralytics.com/models/yolov8/

[4] Zhang Y. и др. ByteTrack Multi Object Tracking by Associating Every Detection Box. https://arxiv.org/abs/2110.06864

[5] Ultralytics. Multi Object Tracking. Документация параметров и конфигурации. https://docs.ultralytics.com/modes/track/

[6] Roboflow Supervision. Каталог демонстрационных видео. https://github.com/roboflow/supervision/blob/0.21.0/supervision/assets/list.py

[7] Ultralytics. Model Training. https://docs.ultralytics.com/modes/train/

[8] Ultralytics. VisDrone Dataset. https://docs.ultralytics.com/datasets/detect/visdrone/

Приложение А Состав экспортируемых данных

Файл	Назначение
annotated.mp4	Видео с рамками, траекториями и счетом
crossings.csv	Кадр, время, ID, класс и направление события
tracks.csv	Покадровые координаты, уверенность и два ID

Файл	Назначение
tracks_summary.csv	Длительность, путь и смещение в пикселях
frame_stats.csv	Число видимых объектов и накопленный счет
summary.json	Параметры запуска и агрегированная статистика
tracker.yaml	Параметры межкадрового сопоставления
progress.json	Последний доступный прогресс обработки

Приложение Б Воспроизведение экспериментов

Команды ниже выполняются из корня проекта в установленном виртуальном окружении. При повторном запуске требуется новое имя эксперимента и новый каталог вывода. Для компьютера без MPS выбирается CPU; время и воспроизводимость могут отличаться.

```
python scripts/download_data.py

python scripts/prepare_visdrone.py --pilot-size 1024

python scripts/run_experiment.py --name vehicle-scratch-repeat --scratch --
epochs 15 --imgsz 416 --batch 8 --device mps

python scripts/run_experiment.py --name vehicle-finetune-repeat --epochs 15 --
imgsz 416 --batch 8 --device mps

python scripts/run_experiment.py --name vehicle-full-repeat --data
configs/visdrone-vehicles.yaml --epochs 10 --imgsz 640 --batch 8 --device mps

python -m unittest discover -s tests -v

python scripts/make_report.py

python -m streamlit run app.py --server.address 127.0.0.1 --server.port 8501
```